



Proposta Técnica: Sistema de Monitoramento e Controle para Hortas Urbanas

Integrantes: Rhyann Portilho, Pedro Shiokawa

Disciplina: Introdução à Computação

Data: [25/04/2026]

1. Estudo de Caso: Diagnóstico e Justificativa

Cenário Real: Moradores de centros urbanos que mantêm pequenas hortas em apartamentos e enfrentam dificuldades no gerenciamento hídrico e climático devido à rotina agitada.

Lacuna de TI Identificada: Falta de um sistema de baixo custo que integre a coleta de dados ambientais em tempo real com a automação inteligente para preservação das plantas.

Justificativa: A implementação de uma infraestrutura de TI permite não apenas a automação (rega), mas a economia de recursos hídricos e a manutenção da saúde das plantas através de dados precisos.

2. Hardware

- **Microcontrolador:** ESP32 DevKit V1 (com Wi-Fi integrado).
 - **Sensor de Solo:** Sensor de Umidade do Solo Capacitivo v1.2 (mais resistente à corrosão que os modelos resistivos).
 - **Sensor de Ar:** DHT11 (Temperatura e Umidade Relativa).
 - **Atuador:** Mini Bomba de Água Submersível (3V-6V) com Mangueira de Silicone de 8mm.
 - **Interface de Potência:** Módulo Relé 5V (1 Canal) para acionamento da bomba.
 - **Alimentação:** Fonte Chaveada 5V/2A com cabo Micro-USB.
-

Redes e Protocolos

- **Protocolo de Transporte:** MQTT (utilizando o broker público **HiveMQ** ou **Adafruit IO**) pela leveza no envio de mensagens.
 - **Segurança:** Autenticação via Token de acesso e criptografia TLS básica suportada pelo ESP32.
-

Software e Ecossistema

- **Desenvolvimento (Firmware):** Arduino IDE ou VS Code com a extensão PlatformIO (Linguagem C++).

- **Serviço de Previsão do Tempo: OpenWeatherMap API** (para obter dados de chuva iminente via JSON).
- **Dashboard de Controle: Blynk IoT ou TagoIO** (permitem criar a interface visual e o aplicativo móvel sem necessidade de programar o front-end do zero).
- **Banco de Dados: InfluxDB** (otimizado para séries temporais de sensores) ou o próprio armazenamento em nuvem do TagoIO.

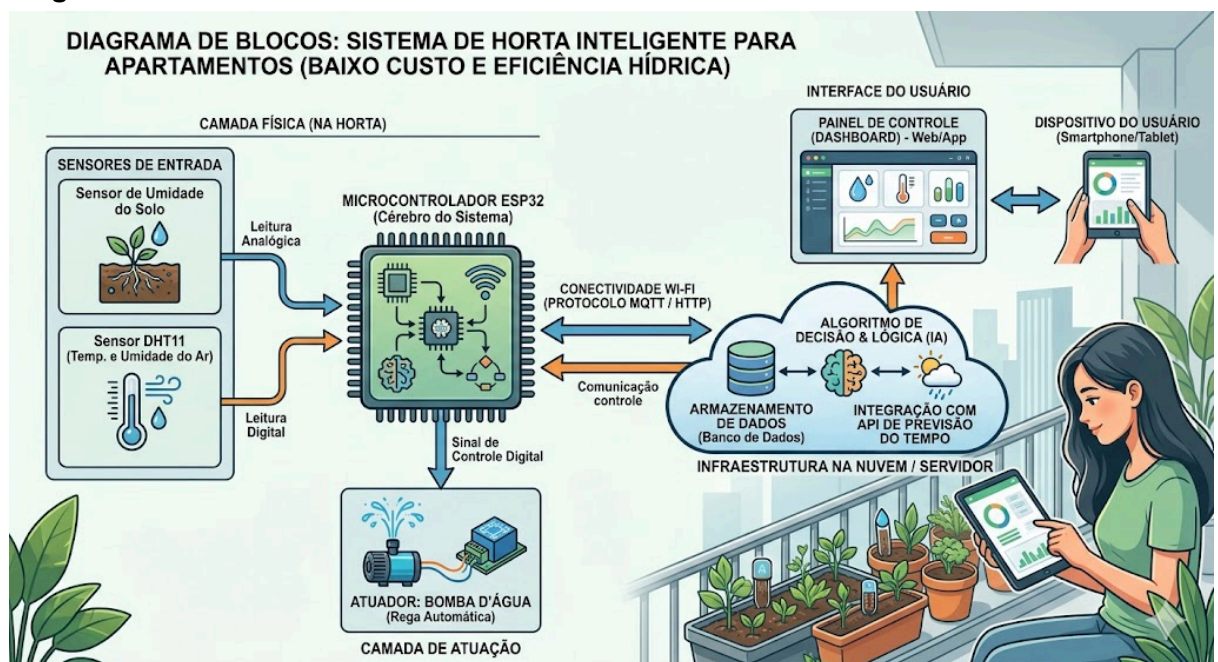
Lógica de Decisão (Algoritmo)

O sistema deve rodar uma rotina de verificação baseada em condicionais lógicas:

1. **Leitura Local:** O ESP 32 lê os sensores de solo e ar a cada 15 minutos.
2. **Consulta Externa:** O sistema faz uma requisição HTTP à API do OpenWeatherMap.
3. **Lógica de Rega:**
 - **Se** a umidade do solo for menor que 30% **E** a previsão de chuva for inferior a 20%: **Acionar bomba.**
 - **Se** a temperatura for superior a 32°C: **Reduzir o intervalo** de verificação para 5 minutos.
 - **Se** a umidade do ar for muito baixa ($<20\%$): **Notificar o usuário** via Dashboard sobre possível estresse hídrico.

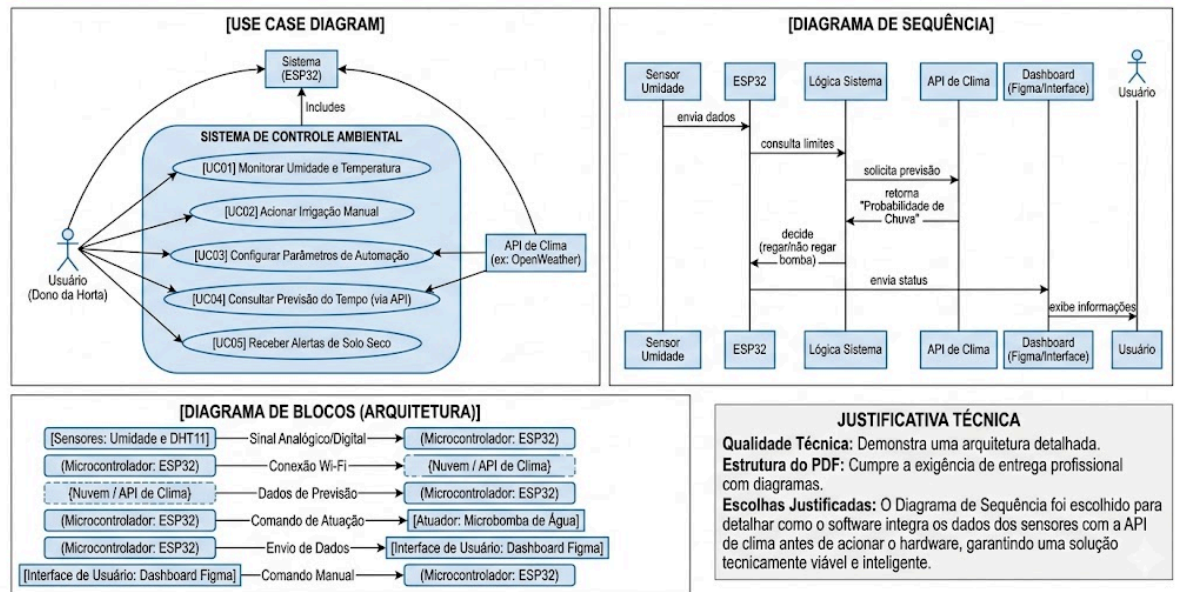
3. Arquitetura do Projeto e Diagramas

- **Diagrama de Blocos:**



- **Diagrama UML (Casos de Uso ou Sequência):**

DIAGRAMAS UML E DE ARQUITETURA PARA SISTEMA DE CONTROLE AMBIENTAL: HORTA INTELIGENTE (ESP32)



- **Justificativa Técnica:**

Hardware: Eficiência e Custo-Benefício

- **ESP32:** Foi selecionado em detrimento do Arduino ou Raspberry Pi devido ao seu **módulo Wi-Fi e Bluetooth nativos**. Em um ambiente de apartamento, a conectividade sem fio é essencial para evitar o cabeamento estruturado. Além disso, o ESP32 possui modos de *Deep Sleep*, permitindo que o sistema funcione por longos períodos com baterias ou fontes de baixa potência.
- **Sensor de Umidade do Solo e DHT11:** O uso do DHT11 oferece uma leitura digital simplificada, eliminando a necessidade de circuitos de conversão complexos. A combinação desses sensores permite uma análise **multivariada**: o sistema não olha apenas se a terra está seca, mas se a temperatura ambiente está alta o suficiente para acelerar a evapotranspiração, antecipando a necessidade de rega.

2. Protocolos de Comunicação (MQTT vs. HTTP)

- **MQTT (Message Queuing Telemetry Transport):** Escolhido por ser um protocolo extremamente leve. Em redes domésticas de apartamentos, onde o sinal de Wi-Fi pode oscilar, o MQTT garante que as mensagens de telemetria (dados dos sensores) cheguem ao servidor com o mínimo de consumo de banda e energia.
- **HTTP/REST:** Utilizado especificamente para a integração com **APIs externas** (como previsão do tempo), facilitando o consumo de dados de servidores globais.

3. Lógica de Software e Integração de Dados

O grande diferencial técnico desta solução não é apenas a automação (regar quando estiver seco), mas a **inteligência preditiva**:

- **Integração com API de Previsão do Tempo:** Esta é a peça-chave para a economia de recursos. Se o sensor detectar solo seco, mas a API informar que a umidade do

ar será de 90% ou que haverá chuva iminente (em hortas de varanda), o algoritmo pode decidir adiar a rega, evitando o desperdício de água e o apodrecimento das raízes.

- **Dashboard em Nuvem:** A centralização dos dados permite que o usuário, em sua rotina agitada, verifique o "status" de saúde da planta em segundos, transformando dados brutos em informação visual acionável.

4. Evidências de Desenvolvimento

- **Repositório GitHub:** [[Repositório GitHub](#)]
- **Simulação Tinkercad:** [Link aqui] — (Circuito funcional e montagem profissional).
- **Protótipo Figma/Canva:** [Link aqui] — (Interface de usuário com alto nível de detalhamento).